OpenAI

OpenAI Status
Compare models | OpenAI API

Opencode/VS Code

OpenCode CRASH Course | OPEN SOURCE AI CODING AGENT
Using Git & GitHub in VSCode: Stage, Commit, and Push

Obsidian

How To Build LLM Wiki In Obsidian? 🧠 A Memory Layer For Any Agentic AI
The Ultimate Obsidian for Beginner's Guide 2025
opencode plugin for obsidian

Bases:
Dataview vs Obsidian Bases: Why (& How) to Make the Switch
obsidian search terms cheat sheet, help.obsidian.md/bases/syntax

Dataview:
Obsidian Dataview Plugin: Beginners Guide and Advanced Workflows
Beginner tutorial for Obsidian Dataview helpful!
dataview info page, dataview beginner guide, Dataview Overview - helpful!

Dataview vs Bases

- Dataview query components
    - Output (required)
        - TABLE
        - LIST
        - TASK
        - CALENDAR
    - Source, FROM
        - Tag
        - Folder
    - Filter, WHERE
        - Define conditions
        - If the value = true it will be displayed
    - Order, SORT
        - Eg reverse chronological
    - Group, GROUP
- Obsidian Bases
    - ``` base \n ``` creates an inline base

- Output
    - Table by default
    - Cards view
    - Can switch back and forth on the same base
    - More are being developed
- Filter, button
    - Conditional statements, you can choose property, statement, and value for each statement
    - Can apply filters for all views or just current view
- Properties, button
    - Select which you want to be visible
- Sort by, button
    - How you want to sort the properties
- Can't group rn but it is on the list for development

Downloading images instructions

Download images locally. In Obsidian Settings → Files and links, set "Attachment folder path" to a fixed directory (e.g. raw/assets/). Then in Settings → Hotkeys, search for "Download" to find "Download attachments for current file" and bind it to a hotkey (e.g. Ctrl+Shift+D). After clipping an article, hit the hotkey and all images get downloaded to local disk. This is optional but useful — it lets the LLM view and reference images directly instead of relying on URLs that may break. Note that LLMs can't natively read markdown with inline images in one pass — the workaround is to have the LLM read the text first, then view some or all of the referenced images separately to gain additional context. It's a bit clunky but works well enough.

LLM wiki resources (ryan's repo)

Andrej Karpathy — "LLM Knowledge Bases" (tweet)

Andrej Karpathy — llm-wiki gist

Hamel Husain & Shreya Shankar — Evals FAQ

A practical FAQ on evaluating LLM applications: error analysis as the highest-value activity, binary pass/fail over fuzzy rating scales, and custom annotation over generic metrics. **How it relates:** this is the backbone of Module 04 (the evaluation loop) — labeling failures and turning them into `AGENTS.md` improvements.

- **Eval is like 80% of the work!** You should always be evaluating and looking at the data of how your system is performing!
- Lots of iterations, quickly = success
- Eval pass rate 70% is the best, much higher than that and its just showing that your eval set is too easy

- Want to evaluate things based on user input
- Re-run error analysis after making significant changes
    - New features
    - Prompt updates
    - Model switches
    - Major bug fixes
- pass/fail grading is usually better than linkert scales (1-5 for ex)
- **A trace**: complete record of all actions, messages, tool calls, and data retrievals from a single initial user query through to the final response. Includes every step across tools, agents, and system components
- **Minimum viable eval set up:**
    - Error analysis > infrastructure
    - *Spend 20 minutes manually reviewing 20-50 LLM outputs whenever you make significant changes*
    - Notebooks to visualize data: youtube.com/watch?v=aqKUwPKBkB0&source_ve_path=OTY3MTQ&embeds_referring_euri=https%3A%2F%2Fhamel.dev%2F
- **Steps**:
    - **Create a dataset:** get representative traces of user interactions with LLM
    - **Open coding:** review and write open-ended notes about traces, noting issues (more qualitative, like journaling)
        - Focus on the first failure that occurs bc upstream errors can cause downstream errors (still note the downstream ones)
    - **Axial coding:** categorize open-ended notes into similar groups, count failures in each group (can use LLM to assist here)
    - **Iterative refinement**: keep iterating on more traces until you reach theoretical saturation where new traces aren't revealing new failures or new info. (goal ~100 traces min, stop after ~20 don't reveal anything new)
    - https://hamel.dev/blog/posts/evals-faq/pawel-error-analysis.png
    - Error Analysis: The Highest ROI Technique In AI Engineering
-

Ryan Lingo — "What Do We Mean by Learning?"

François Chollet — "It's Not About Scale, It's About Abstraction" — https://youtu.be/s7_NlkBwdj8
Chollet argues that scaling models up crushes known benchmarks but doesn't produce genuine intelligence; real intelligence is extracting reusable abstractions and recombining them (the ARC benchmark, deep learning + program synthesis). **How it relates:** the case for staying the critical thinker in the loop — the agent does pattern-matching legwork, but the abstraction, judgment, and acceptance stay yours.

Kenneth Stanley — "Why Greatness Cannot Be Planned" — https://youtu.be/lhYGXYeMq_E
On how rigid objectives can be deceptive, and how open-ended exploration often

reaches places no fixed plan would. **How it relates:** mirrors how a wiki grows — every source you add and question you chase opens the next one; the compounding artifact is something you couldn't have fully specified up front.